

Python 程序设计

第 5 讲 (a): Python 函数的定义与调用

HY Deng

hydeng@shiep.edu.cn



本讲的主要内容

1 1. 为什么要使用函数

2 2. Python 函数的类型

3 3. Python 函数的语法

- Python 函数的定义和调用
- 函数的返回值 return
- 案例学习

什么是 Python 中的函数？

- 1 在 Python 中，函数是一个有名称、组织良好且可复用（DRY——Don't Repeat Yourself，不要重复自己）的代码块，用于完成某一个特定任务.
- 2 由于代码可以复用，函数能提升效率，并减少错误.

Bad Case 1: 复制粘贴式编程

- 多次出现几乎完全相同的代码
- 唯一变化的只是具体数值

```
a = 3  
b = 4  
print(a + b)
```

```
a = 10  
b = 20  
print(a + b)
```

复制粘贴的后果：规模效应

- 如果这种计算出现 **100** 次？
- 修改逻辑 $\Rightarrow$ 改 100 处
- 排查错误 $\Rightarrow$ 检查 100 段代码
- 维护成本随规模 指数级上升

复制粘贴，是程序失控的开始

什么是 Python 中的函数？

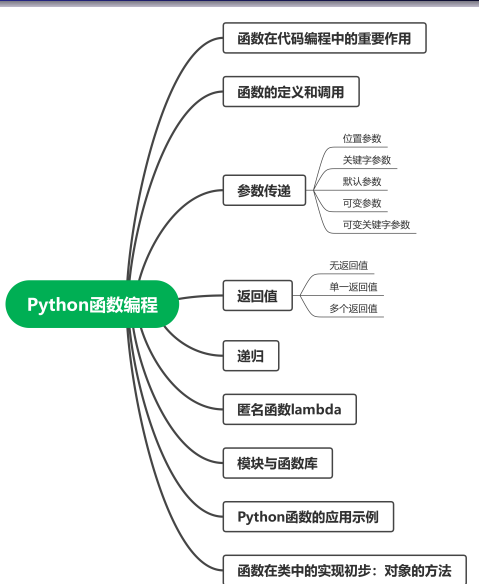
- 3 函数可以帮助我们程序拆分成更小、更模块化的部分.
- 4 拆分后的函数能够让程序结构更加清晰、易于管理.
 - 程序规模不断扩大
 - 功能需求不断变化
 - 相同逻辑反复出现

Python 函数的真正作用

- 抽象：隐藏细节
- 封装：隔离变化
- 复用：避免重复
- 结构化：组织程序

1. 为什么要使用函数
2. Python 函数的类型
3. Python 函数的语法

Python 函数的主要内容



函数的类型（从“哪里来”看）

1 内置函数（Built-in functions）

- 随 Python 解释器一起提供
- 不需要导入，随时可用
- 提供最基础、最通用的功能

2 用户自定义函数（User-defined functions）

- 由程序员根据问题需求自行定义
- 用于封装具体问题的计算逻辑
- 是程序“个性化”和“可复用”的核心

分类标准：函数是 Python 自带的，还是由用户编写的

两类函数的典型示例

内置函数示例:

```
x = -3.5  
print (abs(x))  
n = len ([1, 2, 3, 4])
```

- 功能通用、定义固定
- 由 Python 官方维护
- 使用者只需“调用”，无需关心实现

两类函数的典型示例

用户自定义函数示例:

```
def digit_sum(n):  
    return sum(int(d) for d in str(n))
```

- 功能由问题本身决定
- 由程序员负责设计与维护
- 是解决具体问题的核心工具

常见的内置函数

`range()` # 可以生成一个从指定起始整数到终止整数（不包含终止值）的不可变数字序列

`print()`

`input()`

`type()`

`id()`

`eval()`

初识 range() 函数

```
for i in range(1, 10):  
    print(i, end=' ')
```

输出: 1 2 3 4 5 6 7 8 9

1. 为什么要使用函数
2. Python 函数的类型
3. Python 函数的语法

数学函数: math 包

```
from math import sin
y = sin(pi/2) # 内置函数 sin()
print(y)      # print() 函数
```

类 Class 的方法

```
L = [5, 10, 40, 45]
i = L.index(10)           # i=1
L.append(50)
L.insert(2, 20)
```

本讲的主要内容

1 1. 为什么要使用函数

2 2. Python 函数的类型

3 3. Python 函数的语法

- Python 函数的定义和调用
- 函数的返回值 return
- 案例学习

1. 为什么要使用函数
2. Python 函数的类型
3. Python 函数的语法

Python 函数的定义和调用
函数的返回值 return
案例学习

自定义函数：示例

```
from math import pi  
  
def circle_area(r):  
    S = pi * r ** 2  
    return S
```

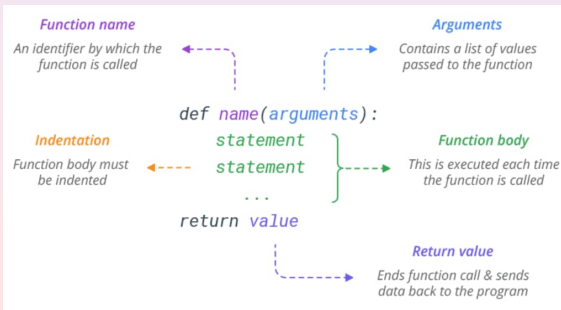
def关键字：定义函数

函数输入参数

return关键字：设置返回值

自定义函数：语法

```
def function_name(parameter1, parameter2):  
    """docstring"""  
    # function body  
    # write some action  
    return value
```



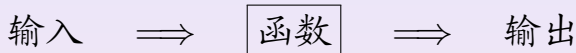
自定义函数：语法

- 1 **def**关键字：用于标识函数定义的开始
- 2 **function-name** 函数名：用于唯一标识函数，命名规则遵循 Python 标识符命名规则
- 3 **parameter**：传递给函数的参数（可选），函数可以接收零个或多个参数
- 4 **:**（**colon**，冒号）：标志函数头的结束

自定义函数：语法

- 5 函数体 **function body**: 执行具体任务的代码块，所有语句必须保持相同的缩进级别
- 6 **"""docstring"""** 文档字符串：用于描述函数的功能
- 7 **return**: 结束函数，选择性地返回一个值或多个值给调用方。若**return**后不带表达式，则返回**None**

如何理解一个函数？



- 输入：函数需要什么？
- 功能：函数做什么？
- 输出：函数给出什么？

Python 中函数的三要素

- 参数 (**Parameters**): 输入
- 函数体 (**Body**): 计算过程
- **return**: 输出

return决定了函数“算出了什么”

一个最基本的函数

```
def add(a, b):  
    c = a + b  
    return c
```

- 输入: a, b
- 内部计算: $a + b$
- 输出: c

本讲的主要内容

1 1. 为什么要使用函数

2 2. Python 函数的类型

3 3. Python 函数的语法

- Python 函数的定义和调用
- 函数的返回值 return
- 案例学习

函数：无返回值

```
# 无返回值  
  
def print_welcome(name):  
    print(f"Welcome {name}")  
  
# 函数调用  
print_welcome("SUEP")
```

输出结果：Welcome SUEP

函数返回: None

```
def add(a, b):  
    print(a + b)
```

```
x = add(3, 4)  
print(x)
```

x 的值是 None

为什么会返回 None?

- Python 的规则:
 - 没有写 return
 - 等价于 return None
- 含义:
 - 函数“执行了”
 - 但“没有计算结果”

一个很多人没意识到的事实

`return` = 返回结果 + 立刻结束函数

- `return` 之后的代码
- 永远不会执行

return 会立刻终止函数

```
def test():  
    print("A")  
    return  
    print("B")  
  
test()
```

只会打印 A

函数不是“顺序执行到最后”

- 脚本程序：
 - 从上到下
 - 执行到文件末尾
- 函数：
 - 执行到 return
 - 或执行到函数末尾

函数有明确的“结束点”

一个函数可以有多个 return 吗？

多个 return？

可以，而且非常常见

- return 是控制流工具
- 不只是“最后一句”

分支中的 return

```
def sign(x):  
    if x > 0:  
        return 1  
    elif x < 0:  
        return -1  
    else:  
        return 0
```

- 不同条件
- 不同出口

函数的执行路径

- 函数调用后:
 - 只会走一条分支
 - 一旦 return, 函数结束

一次调用, 只可能执行一个 return

常见误解

■ 错误想法:

- “if 里的 return 执行完，还会继续往下”

■ 事实:

- return 立即结束整个函数

return 不是 break

工程中非常常见的写法

```
def withdraw(balance, amount):  
    if amount <= 0:  
        return "Invalid amount"  
    if amount > balance:  
        return "Insufficient balance"  
    return balance - amount
```

- 错误情况先 return
- 正常逻辑最后 return

自定义函数：返回多个值

例 1：计算长方形的周长和面积

$$C = 2(a + b), \quad S = ab$$

```
def rect(width, height):  
    C = 2 * (width + height) # 计算周长  
    S = width * height      # 计算面积  
    return C, S  
  
#调用函数  
w = 4 # 宽度  
h = 5 # 高度  
C, S = rect(w, h)  
print(f'周长为: {C}, 面积为: {S}')
```

输出： 周长为： 18， 面积为： 20

本讲的主要内容

1 1. 为什么要使用函数

2 2. Python 函数的类型

3 3. Python 函数的语法

- Python 函数的定义和调用
- 函数的返回值 return
- 案例学习

案例 1: 无 return 的函数

```
def greet(name):  
    print(f"Hello , {name}!")  
  
greet("Alice")  
greet("Bob")
```

- 函数用于“执行一件事情”
- 不产生返回值
- 调用后立即产生输出效果

适合：打印、提示、日志等操作

案例 2: 有 return 的函数

```
def square(x):  
    return x * x
```

```
a = square(3)
```

```
b = square(5)
```

```
print(a, b)
```

- 函数用于“计算一个结果”
- return 把结果交还给调用者
- 返回值可以被赋值、参与后续计算

这是最常见、最重要的函数形式

案例 3: return 多个值

```
def min_max(a, b):  
    if a < b:  
        return a, b  
    else:  
        return b, a  
x, y = min_max(10, 3)  
print(x, y)
```

- 一个函数可以返回多个结果
- 实际返回的是一个元组 (tuple)
- 调用时可用多个变量接收

非常适合“同时计算多个相关结果”

如何选择函数形式?

■ 无 return

- 侧重动作和过程
- 如: 打印、提示、更新状态

■ 有 return

- 侧重结果和计算
- 如: 数值计算、判断逻辑

■ return 多个值

- 返回一组相关结果
- 提高函数表达能力

1. 为什么要使用函数
2. Python 函数的类型
3. Python 函数的语法

Python 函数的定义和调用
函数的返回值 return
案例学习

Thanks!